

CampusEats

Part C · Caching & Safe Retries

Based on: CampusEats HTTP Methods & Headers assignment, B4 Cache a Read, Assignment 4 C7 Idempotency-Key, and A3 Safe & Idempotent Endpoints.

Service context: This document uses the CampusEats order endpoints supplied for the assignment context, especially GET /orders/{orderId}, POST /orders, PUT /orders/{orderId}, and POST /orders/{orderId}/checkout.

1. C1 — Conditional GET → 304 Not Modified

The single-item read endpoint GET /orders/{orderId} uses the ETag mechanism introduced in B4. The server returns an ETag representing the current version of the order. The ETag must change whenever the represented order resource changes.

If the client already has the same ETag, it sends If-None-Match. When the supplied ETag still matches the current ETag, the server returns **304 Not Modified** with no response body.

```
Request
GET /orders/42 HTTP/1.1
Host: campuseats.example
If-None-Match: "order-v12"

Response
HTTP/1.1 304 Not Modified
ETag: "order-v12"
Cache-Control: private, max-age=60
```

Purpose: The client can continue using its cached representation instead of downloading the same order JSON again. This reduces unnecessary response data while preserving freshness through the ETag.

2. C2 — Conditional Write → 412 Precondition Failed

For an update endpoint such as PUT /orders/{orderId}, the client sends If-Match with the ETag version it previously read. The server updates the order only when the supplied ETag matches the current ETag.

If the resource has changed since the client read it, the ETags do not match. The server must return **412 Precondition Failed** and must not overwrite the newer representation.

```
Current resource:
ETag: "order-v12"

Request
PUT /orders/42 HTTP/1.1
Host: campuseats.example
If-Match: "order-v11"
Content-Type: application/json

{
  "status": "CONFIRMED"
}

Response
HTTP/1.1 412 Precondition Failed
ETag: "order-v12"
Content-Type: application/json

{
  "error": "Precondition Failed",
  "message": "Resource has changed; refresh before updating."
}
```

Purpose: This prevents two clients/editors from overwriting each other's changes using a stale representation.

3. C3 — Idempotency-Key

Assignment 4 C7 identifies order creation and checkout as operations where duplicate work can cause real damage. The chosen implementation protects POST /orders with an Idempotency-Key.

The server stores the idempotency key together with the created order/result. If the same key is received again, the server returns the original result instead of creating another order or repeating the work.

```
First request
POST /orders HTTP/1.1
Host: campuseats.example
Idempotency-Key: order-create-abc123
Content-Type: application/json
```

```
{
  "userId": 17,
  "restaurantId": 5,
  "items": [
    { "menuItemId": 21, "quantity": 2 }
  ]
}
```

```
Response
HTTP/1.1 201 Created
Location: /orders/42
Content-Type: application/json
```

```
{
  "orderId": 42,
  "status": "PENDING"
}
```

```
Retry with the SAME Idempotency-Key
POST /orders HTTP/1.1
Host: campuseats.example
Idempotency-Key: order-create-abc123
Content-Type: application/json
```

```
{ same request body }
```

```
Response
HTTP/1.1 201 Created
Location: /orders/42
Content-Type: application/json
```

```
{
  "orderId": 42,
  "status": "PENDING"
}
```

Important: The retry must return the original result and must not create order 43. A duplicate order could lead to duplicate order processing and potentially duplicate downstream payment/delivery work.

5. ETag Versioning Example — Implementation & Output

Express.js implementation of conditional GET using ETag versioning:

```
const express = require("express");

const app = express();
const PORT = 3000;

let order = {
  orderId: 42,
  status: "PENDING",
  version: 1
};

// GET /orders/:id - Supports conditional GET with ETag
app.get("/orders/:id", (req, res) => {
  if (req.params.id !== "42") {
    return res.status(404).json({ error: "Order not found" });
  }

  const currentETag = `order-v${order.version}`;

  if (req.headers["if-none-match"] === currentETag) {
    return res.status(304)
      .set("ETag", currentETag)
      .set("Cache-Control", "private, max-age=60")
      .end();
  }

  res.status(200)
    .set("ETag", currentETag)
    .set("Cache-Control", "private, max-age=60")
    .json({
      orderId: order.orderId,
      status: order.status
    });
});

// PUT /orders/42 - Updates status and increments version
app.put("/orders/42", (req, res) => {
  order.status = "CONFIRMED";
  order.version++;

  res.json({
    message: "Order updated",
    order
  });
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Output:

```
PS C:\CollegeLab_2ndSem\Web_Services\EtagDemo> curl.exe -i http://localhost:3000/orders/42
HTTP/1.1 200 OK
X-Powered-By: Express
ETag: "order-v1"
Cache-Control: private, max-age=60
Content-Type: application/json; charset=utf-8
Content-Length: 33
Date: Tue, 22 Sep 2026 16:35:20 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Mechanism	Prevents / saves
ETag + If-None-Match	Avoids sending an unchanged representation. A matching validator produces 304 with no body.
ETag + If-Match	Prevents a stale client from overwriting a resource that changed after it was read.
Idempotency-Key	Prevents duplicate work when a client retries a non-idempotent operation such as order creation or checkout.

7. Implementation Checklist for Part C

- ✓ C1: Add/verify If-None-Match handling on the B4 single-item GET and return 304 with no body when the ETag matches.
- ✓ C2: Add/verify If-Match handling on the update endpoint and return 412 without changing the resource when the ETag does not match.
- ✓ C3: Store Idempotency-Key with the result of the protected operation and return the original result for a repeated key without doing duplicate work.
- ✓ C4: Include the safe/idempotent classification and retry mechanism table in NOTES.md.
- ✓ Add tests for 304, 412, and repeated Idempotency-Key behavior.
- ✓ Reflect the new request headers, response statuses, and behavior in `openapi.yaml`.
- ✓ Later, Part D should demonstrate the successful create, idempotent repeat, 304, 412, and required failure cases using `curl -v`.